

LD Camera, Camera Controls, and Animation

High-level usage documentation for London Dynamics extensions on `<model-viewer>`.

LD Camera
Save, restore, reset, rotate, truck, and switch camera type.

LD Camera Controls
Switch between orbit and FPS controls, keyboard movement, fly mode, and viewport gizmo.

LD Animation
Animate between camera views with easing and optional subject avoidance.

1. Mental model

The LD camera APIs split responsibility into three layers:

- **Camera pose:** where the camera is, what it looks at, and its field of view.
- **Camera control mode:** how users interact after the view is applied: `orbit` or `fps`.
- **Camera transition:** whether the pose is applied immediately or animated over time.

Use `setCameraControlsMode()` when only the interaction mode changes. Use `setCameraView()` when applying a saved view immediately. Use `animateCameraTo()` when moving to a saved view with animation.

2. LD Camera Controls

LD controls add FPS mode and an optional viewport gizmo on top of the standard camera controls.

Attribute	JS property	Default	Description
<code>camera-control-mode</code>	<code>cameraControlMode</code>	<code>orbit</code>	Selects <code>orbit</code> or <code>fps</code> behavior.
<code>fps-keyboard-move</code>	<code>fpsKeyboardMove</code>	<code>false</code>	Enables keyboard movement in FPS mode.
<code>fps-fly-mode</code>	<code>fpsFlyMode</code>	<code>false</code>	When true, FPS forward/back movement follows camera pitch. When false, movement stays level.
<code>fps-look-sensitivity</code>	<code>fpsLookSensitivity</code>	<code>0.5</code>	Pointer look sensitivity in FPS mode.
<code>fps-move-sensitivity</code>	<code>fpsMoveSensitivity</code>	<code>0.3</code>	Keyboard movement speed multiplier in FPS mode.
<code>viewport-gizmo</code>	<code>showViewportGizmo</code>	<code>false</code>	Shows an interactive orientation cube to snap the camera to common views.

Control mode method

```
modelViewer.setCameraControlsMode('fps', {
  enableKeyboardMove: true,
  enableFlyMode: true,
});
```

```
modelViewer.setCameraControlsMode('orbit');
```

In FPS mode, users drag to look around. With keyboard movement enabled, `W/A/S/D` and arrow keys move horizontally, `E` moves up, and `Q` moves down.

3. LD Camera views

A camera view can be attribute-style or JSON-style.

Attribute-style view

```
const outsideView = {
  controlMode: 'orbit',
  cameraOrbit: '-120deg 78deg 7m',
  cameraTarget: '0m 0.45m 0m',
  fieldOfView: '35deg',
};
```

JSON-style view

Use `getCameraJSON()` to capture the current camera. Store the returned `object`, then pass it to `setCameraFromJSON()`, `setCameraView()`, or `animateCameraTo()`.

```
const saved = modelViewer.getCameraJSON();
localStorage.setItem('camera', JSON.stringify(saved.object));

const restored = JSON.parse(localStorage.getItem('camera'));
await modelViewer.setCameraView(restored);
```

View fields

Field	Description
<code>controlMode</code>	Optional <code>orbit</code> or <code>fps</code> . If present, the view also changes controls mode.
<code>cameraOrbit</code>	Optional orbit string such as <code>-120deg 78deg 7m</code> .
<code>cameraTarget</code>	Optional target string such as <code>0m 0.45m 0m</code> .
<code>fieldOfView</code>	Optional FOV string such as <code>35deg</code> .
<code>enableKeyboardMove</code>	Optional FPS keyboard movement override.
<code>enableFlyMode</code>	Optional FPS fly mode override.
<code>position</code> , <code>quaternion</code> , <code>matrix</code> , <code>target</code> , <code>worldTarget</code> , <code>fov</code>	JSON-style camera fields emitted by <code>getCameraJSON()</code> .
<code>controlsSnapshot</code>	Recommended saved CameraControls state for reliable restore after pan/truck.

4. LD Camera methods

Method	Use when	Description
<code>getCameraJSON()</code>	Saving a view	Returns <code>{ metadata, object }</code> . Persist <code>object</code> .
<code>setCameraFromJSON(camera)</code>	Restoring exact saved camera data	Restores a camera object produced by <code>getCameraJSON()</code> .
<code>setCameraView(view, options?)</code>	Jumping to a named view	Applies a view immediately. If the view includes <code>controlMode</code> , it also changes controls mode.
<code>animateCameraTo(view, options?)</code>	Moving between views smoothly	Animates from the current pose to the target view and can apply the target <code>controlMode</code> .
<code>resetCamera()</code>	Returning to saved/default controls state	Stops active controls motion and resets the camera controls.
<code>rotateCamera(azimuth, polar, animate?)</code>	Rotating programmatically	Rotates by degree values; optionally animates.
<code>truckCamera(x, y, z?)</code>	Programmatic pan/truck	Moves the CameraControls look-at target, similar to user pan.
<code>setCurrentAsDefaultCamera()</code>	Updating reset baseline	Saves the current camera controls state as the default.
<code>setCameraType(type)</code>	Switching projection	Sets <code>perspective</code> or <code>orthographic</code> .
<code>getCameraType()</code>	Reading projection	Returns <code>perspective</code> or <code>orthographic</code> .
<code>toggleCameraType()</code>	UI projection toggle	Switches between perspective and orthographic.

5. Camera animation

`animateCameraTo(view, options?)` is the main LD animation API for camera movement.

```
await modelViewer.animateCameraTo(
  {
    controlMode: 'orbit',
    cameraOrbit: '-120deg 78deg 7m',
    cameraTarget: '0m 0.45m 0m',
    fieldOfView: '35deg',
  },
  {
    duration: 1400,
    easing: 'easeInOutQuad',
    avoidSubject: true,
    avoidMargin: 0,
  }
)
```

```
    }  
  );  
};
```

Option	Default	Description
<code>duration</code>	<code>300</code>	Animation duration in milliseconds. Use <code>0</code> to apply the end pose immediately.
<code>easing</code>	<code>easeInOutQuad</code>	Easing curve name. Supported names include <code>linear</code> , sine, quad, cubic, quart, quint, expo, circ, back, elastic, and bounce variants.
<code>avoidSubject</code>	<code>false</code>	Routes animation around model bounds when the straight path intersects the subject.
<code>avoidMargin</code>	<code>0</code>	Extra margin around the subject avoidance bounds.

Important: If the target view includes `controlMode: 'fps'`, the animation can end in FPS mode. If it includes `controlMode: 'orbit'`, it ends in orbit mode.

6. Recommended patterns

Named camera slots

```
const cameraSlots = {  
  outside: {  
    controlMode: 'orbit',  
    cameraOrbit: '-120deg 78deg 7m',  
    cameraTarget: '0m 0.45m 0m',  
    fieldOfView: '35deg',  
  },  
  inside: {  
    ...savedCameraObject,  
    controlMode: 'fps',  
    enableKeyboardMove: true,  
    enableFlyMode: true,  
  },  
};  
  
await modelViewer.setCameraView(cameraSlots.inside);
```

Animated tour step

```
async function goToNextView(view) {  
  await modelViewer.animateCameraTo(view, {  
    duration: 1200,  
    easing: 'easeInOutQuad',  
    avoidSubject: true,  
  });  
}
```

Mode-only toggle

```
function enableWalkthroughMode() {  
  modelViewer.setCameraControlsMode('fps', {  
    enableKeyboardMove: true,  
    enableFlyMode: false,  
  });  
}
```

Generated for the London Dynamics model-viewer fork. Focused on LD Camera, LD Camera Controls, and camera animation APIs.